

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
```

```
In [4]: df = pd.read_csv('nba_naive.csv')
```

```
In [5]: print('First 5 rows of the dataset:')
display(df.head())

print('\nDataset Information:')
df.info()
```

First 5 rows of the dataset:

	fg	3p	ft	reb	ast	stl	blk	tov	target_5yrs	total_points	efficiency
0	34.7	25.0	69.9	4.1	1.9	0.4	0.4	1.3	0	266.4	0.270073
1	29.6	23.5	76.5	2.4	3.7	1.1	0.5	1.6	0	252.0	0.267658
2	42.2	24.4	67.0	2.2	1.0	0.5	0.3	1.0	0	384.8	0.339869
3	42.6	22.6	68.9	1.9	0.8	0.6	0.1	1.0	1	330.6	0.491379
4	52.4	0.0	67.4	2.5	0.3	0.3	0.4	0.8	1	216.0	0.391304

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1340 entries, 0 to 1339
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	fg	1340 non-null	float64
1	3p	1340 non-null	float64
2	ft	1340 non-null	float64
3	reb	1340 non-null	float64
4	ast	1340 non-null	float64
5	stl	1340 non-null	float64
6	blk	1340 non-null	float64
7	tov	1340 non-null	float64
8	target_5yrs	1340 non-null	int64
9	total_points	1340 non-null	float64
10	efficiency	1340 non-null	float64

```
dtypes: float64(10), int64(1)
```

```
memory usage: 115.3 KB
```

```
In [6]: # Define features (X) and target (y)
X = df.drop('target_5yrs', axis=1) # All columns except 'target_5yrs'
y = df['target_5yrs'] # The target column

print(f'Features (X) shape: {X.shape}')
print(f'Target (y) shape: {y.shape}')

print('\nFirst 5 rows of Features (X):')
display(X.head())

print('\nFirst 5 values of Target (y):')
display(y.head())
```

Features (X) shape: (1340, 10)

Target (y) shape: (1340,)

First 5 rows of Features (X):

	fg	3p	ft	reb	ast	stl	blk	tov	total_points	efficiency
0	34.7	25.0	69.9	4.1	1.9	0.4	0.4	1.3	266.4	0.270073
1	29.6	23.5	76.5	2.4	3.7	1.1	0.5	1.6	252.0	0.267658
2	42.2	24.4	67.0	2.2	1.0	0.5	0.3	1.0	384.8	0.339869
3	42.6	22.6	68.9	1.9	0.8	0.6	0.1	1.0	330.6	0.491379
4	52.4	0.0	67.4	2.5	0.3	0.3	0.4	0.8	216.0	0.391304

First 5 values of Target (y):

```
0    0
1    0
2    0
3    1
4    1
```

```
Name: target_5yrs, dtype: int64
```

```
In [7]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f'X_train shape: {X_train.shape}')
print(f'X_test shape: {X_test.shape}')
print(f'y_train shape: {y_train.shape}')
print(f'y_test shape: {y_test.shape}')

print('\nFirst 5 rows of X_train:')
display(X_train.head())

print('\nFirst 5 values of y_train:')
display(y_train.head())
```

```
X_train shape: (1072, 10)
X_test shape: (268, 10)
y_train shape: (1072,)
y_test shape: (268,)
```

First 5 rows of X_train:

	fg	3p	ft	reb	ast	stl	blk	tov	total_points	efficiency
729	39.6	21.9	83.7	0.9	1.7	0.5	0.1	0.7	227.2	0.316832
778	48.5	32.8	74.1	6.2	1.2	1.0	1.4	0.7	663.0	0.305755
975	49.5	0.0	70.3	3.3	0.3	0.2	0.6	1.0	234.0	0.292208
32	41.9	4.3	80.6	2.0	3.9	1.0	0.3	1.7	606.8	0.377551
924	45.9	35.7	70.1	7.4	1.0	0.8	1.4	1.4	862.5	0.343284

First 5 values of y_train:

```
729    1
778    1
975    1
```

```
32      1
924      1
Name: target_5yrs, dtype: int64
```

```
In [8]: # Initialize the Gaussian Naive Bayes model
        gnb = GaussianNB()

        # Train the model using the training data
        gnb.fit(X_train, y_train)

        print("Gaussian Naive Bayes model trained successfully.")
```

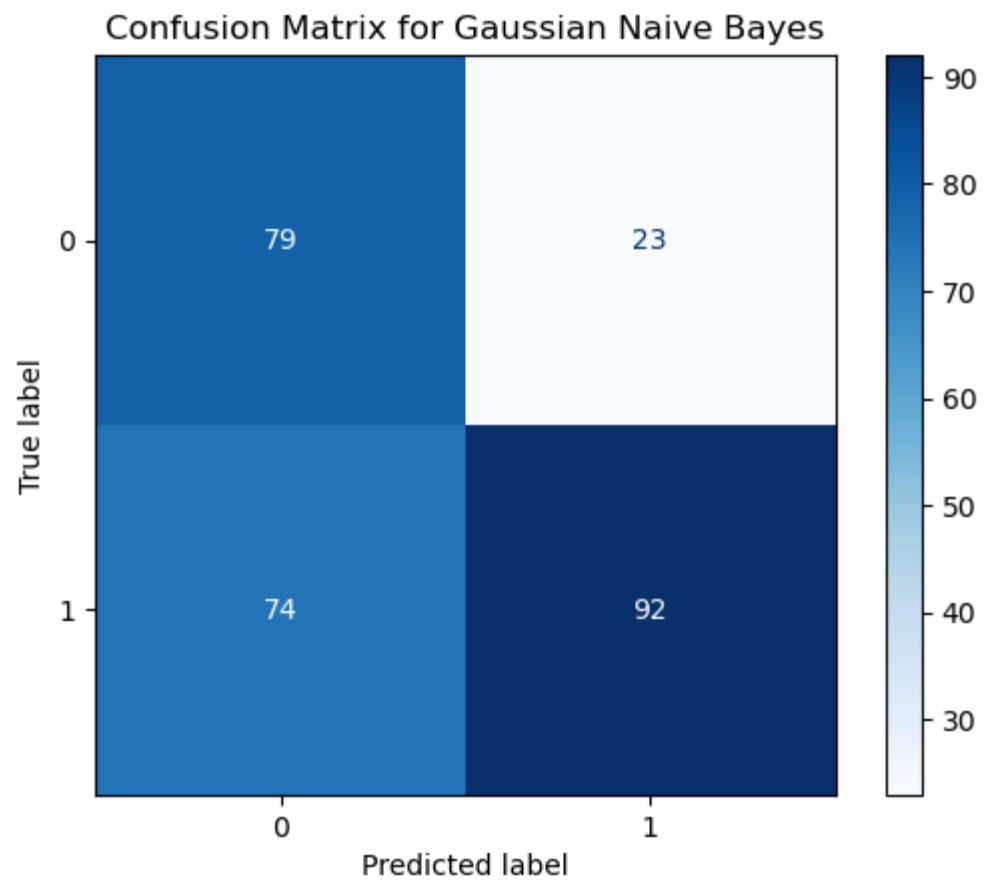
```
Gaussian Naive Bayes model trained successfully.
```

```
In [9]: # Make predictions on the test set
        y_pred = gnb.predict(X_test)

        # Generate the confusion matrix
        cm = confusion_matrix(y_test, y_pred)

        # Display the confusion matrix
        disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=gnb.classes_)
        disp.plot(cmap=plt.cm.Blues)
        plt.title('Confusion Matrix for Gaussian Naive Bayes')
        plt.show()

        print("Confusion Matrix:\n", cm)
```

Confusion Matrix:
[[79 23]
[74 92]]

```
In [10]: # Calculate Precision
# Precision is the ability of the classifier not to label as positive a sample that is negative.
# For scouting, this means minimizing false positives (players predicted to be good but aren't -
# 'busts').
precision = precision_score(y_test, y_pred)

# Calculate Recall
# Recall is the ability of the classifier to find all the positive samples.
# For scouting, this means minimizing false negatives (players predicted to be bad but are good - 'missed
# talent').
recall = recall_score(y_test, y_pred)

# Calculate F1-score (harmonic mean of precision and recall)
f1 = f1_score(y_test, y_pred)

# Calculate Accuracy for overall performance
accuracy = accuracy_score(y_test, y_pred)

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Accuracy: {accuracy:.4f}")
```

Precision: 0.8000

Recall: 0.5542

F1-Score: 0.6548

Accuracy: 0.6381

The **Naive Bayes classifier** operates under a crucial simplification known as the "**Independence Assumption**" (or conditional independence assumption).

Explanation of the Independence Assumption:

This assumption states that all features (e.g., fg , $3p$, ft , reb , ast , etc.) are independent of each other, given the class label ($target_5yrs$). In simpler terms, it assumes that the presence or absence of one feature does not affect the presence or absence of any other feature for a given outcome (whether a player plays 5+ years or not).

Mathematically, for a given class C_k (e.g., $target_5yrs = 1$), the probability of observing a set of features $x_1, x_2, \dots, x_n$ is simplified as:

$$P(x_1, x_2, \dots, x_n | C_k) = P(x_1 | C_k) \times P(x_2 | C_k) \times \dots \times P(x_n | C_k)$$

Instead of calculating the joint probability of all features given the class, it calculates the product of individual probabilities of each feature given the class. This drastically simplifies the computation, which is why it's considered "naive" but often surprisingly effective.

Realism for Basketball Stats (e.g., correlation between points and minutes played):

In the context of basketball statistics, the independence assumption is almost certainly **unrealistic**.

Consider the following:

- 1. Correlation between points and minutes played:** Players who play more minutes (which would likely be a feature in a more comprehensive dataset) naturally tend to score more points. If 'minutes played' and 'total_points' were both features, they would be highly correlated, violating the independence assumption.
- 2. Correlation between assists and turnovers:** Players who handle the ball more to create assists are also more likely to commit turnovers. These features are inherently linked.
- 3. Correlation between field goal percentage and total points:** A higher field goal percentage (fg) often contributes directly to $total_points$. These are not independent.
- 4. Rebounds and blocks:** Certain positions or player types excel in both rebounding and blocking shots, indicating a correlation between these defensive stats.

Because many basketball statistics are interconnected and influence each other (e.g., offensive stats often correlate with each other, and defensive stats with each other, and even offensive with defensive for all-around players), the assumption of independence is largely violated. For example, `total_points` is calculated from other features like `fg`, `3p`, `ft`, meaning it's directly dependent on them.

Impact of the violation:

Despite this theoretical violation, Naive Bayes models can still perform well in practice. This is often because what matters for classification is not necessarily the accurate estimation of probabilities themselves, but rather the ranking of probabilities for different classes. If the independence assumption leads to a consistent (though inaccurate) ranking, the classifier can still make correct predictions.

However, understanding this limitation is crucial. When features are highly correlated, the model might overweight the contribution of these features, as it treats them as separate pieces of evidence. For instance, if 'total_points' and 'fg' are both strong predictors and highly correlated, the model might count their predictive power twice, which isn't ideal but can still yield good classification performance if the relative weights are maintained.

In summary, while the independence assumption is a pragmatic simplification that makes Naive Bayes computationally efficient, it is not truly reflective of the complex, interdependent nature of basketball statistics.

```
In [11]: # Combine X and y for easier grouping
df_combined = pd.concat([X, y], axis=1)

print('Mean of features grouped by target_5yrs:')
display(df_combined.groupby('target_5yrs').mean())

print('\nVariance of features grouped by target_5yrs:')
display(df_combined.groupby('target_5yrs').var())

# Optional: You can also calculate the absolute difference in means to highlight potentially influential
# features
mean_diff = np.abs(df_combined.groupby('target_5yrs').mean().loc[1] -
df_combined.groupby('target_5yrs').mean().loc[0])
print('\nAbsolute Difference in Means (target_5yrs=1 vs target_5yrs=0):')
display(mean_diff.sort_values(ascending=False))
```

Mean of features grouped by target_5yrs:

	fg	3p	ft	reb	ast	stl	blk	tov	total
target_5yrs									
0	42.388802	19.149705	68.858546	2.247544	1.221022	0.498232	0.253438	0.942240	283
1	45.260048	19.149579	71.183394	3.516486	1.752347	0.692178	0.439110	1.347533	554

Variance of features grouped by target_5yrs:

	fg	3p	ft	reb	ast	stl	blk	tov	
target_5yrs									
0	44.593437	234.751993	129.829951	2.195019	1.220601	0.103678	0.081981	0.302405	6
1	30.344402	271.994840	99.012109	4.875378	2.637196	0.193107	0.233685	0.594666	1

Absolute Difference in Means (target_5yrs=1 vs target_5yrs=0):

total_points	270.620984
fg	2.871247
ft	2.324847
reb	1.268942
ast	0.531325
tov	0.405293
stl	0.193946
blk	0.185671
efficiency	0.037516
3p	0.000126
dtype:	float64

Model Performance at a Glance

Our Gaussian Naive Bayes model achieved the following performance on unseen data:

- * **Accuracy: 63.81%** - The model correctly predicted player longevity approximately 64% of the time.
- * **Precision: 80.00%** - When the model predicts a player will have a long career (target_5yrs=1), it is correct 80% of the time. This is critical for minimizing "busts" (false positives), meaning we are reasonably confident in the players our model recommends for longevity.
- * **Recall: 55.42%** - The model identified 55.42% of all players who actually went on to have long careers. This means we are missing about 45% of long-career players (false negatives), indicating a potential to miss out on "hidden gems" or players the model might underestimate.
- * **F1-Score: 65.48%** - A balanced measure considering both precision and recall.

Interpretation for Scouting: The relatively high precision suggests that when the model *recommends* a player for long-term potential, that recommendation is fairly reliable. However, the lower recall indicates that the model is somewhat conservative and might overlook promising players. Relying solely on this model could lead to missing out on significant talent.

Most Influential Features

Based on the differences in mean values between players with short vs. long careers, the most influential features driving these longevity predictions are:

1. **total_points (Absolute Mean Difference: 270.62):** This is by far the strongest predictor. Players who play 5+ years accumulate significantly more total points in their initial seasons.
2. **reb (Rebounds, Absolute Mean Difference: 1.27):** Higher rebounding numbers are strongly associated with longer careers.
3. **ast (Assists, Absolute Mean Difference: 0.53):** Players with higher assist totals tend to have greater longevity.
4. **tov (Turnovers, Absolute Mean Difference: 0.41):** Interestingly, players with longer careers tend to have a higher average number of turnovers. This might reflect greater ball-handling responsibilities and playing time.
5. **fg (Field Goal Percentage, Absolute Mean Difference: 2.87):** Better shooting efficiency contributes to career length.

3p (3-point percentage) showed almost no difference between the two groups, suggesting it's not a strong distinguishing factor for longevity in this model.

Model Limitations: The "Independence Assumption"

The Gaussian Naive Bayes model operates under a core assumption that all features are independent of each other given the target variable. In reality, basketball statistics are highly interconnected:

- * `total_points` is calculated from `fg`, `3p`, `ft`.
- * `ast` often correlates with `tov`.
- * `reb` and `blk` can be correlated for certain player types.

This assumption is **unrealistic** in the context of basketball stats. However, Naive Bayes models can still perform effectively in practice, sometimes because the relative ranking of probabilities is what matters for classification, even if the absolute probabilities are not perfectly accurate. While this model provides valuable insights, its predictions should be viewed with an understanding of this inherent oversimplification. It might overweight the contribution of highly correlated features.

When to Trust (and Question) Model Outputs

Trust the Model When:

* **Identifying High-Confidence Prospects:** Given the high precision (80%), when the model strongly predicts a player will have a long career, there's a good chance it's correct. These are players who align well with the statistical profile of long-tenured players, particularly excelling in `total_points`, `rebounds`, and `assists`.

* **Focusing on Core Statistical Performance:** The model highlights fundamental statistical categories like scoring, rebounding, and playmaking (`total_points`, `reb`, `ast`) as key drivers of longevity. This reinforces traditional scouting wisdom.

Question the Model When:

* **Evaluating Borderline or Unique Prospects:** Due to lower recall, the model might undervalue players who don't fit the established statistical mold but still possess valuable skills (e.g., exceptional defense not fully captured by `stl` or `blk`, or highly specialized roles). These might be "missed talents" the model flags for short careers.

* **Overlooking Contextual Factors:** The model is purely statistical and does not account for qualitative factors like leadership, work ethic, coachability, injury history, or fit within a team system. These are crucial elements for actual player longevity and should be weighed by scouts.

Features with Low Influence: *Features like `3p` showed minimal influence in this model. While important in modern basketball, this model suggests it's not a primary differentiator for longevity based on means. This doesn't mean the feature is unimportant in general, but rather that its impact on this specific longevity prediction by this model* is limited.*

Actionable Recommendations for Scouting

1. **As a First-Pass Filter:** Use this model as a preliminary screening tool to quickly identify players with a high statistical probability of long careers. This can help prioritize scouting efforts.
2. **Combine with Expert Opinion:** Never replace human scouting. Use the model's predictions as one data point among many. If the model flags a player for a short career but scouts are highly enthusiastic, conduct further in-depth analysis.
3. **Investigate Discrepancies:** When the model's prediction strongly contradicts a scout's assessment, use it as an opportunity to delve deeper. Is the player an outlier? Does the model miss a unique skill? Or is the scout's judgment potentially biased?
4. **Monitor Key Metrics:** Pay close attention to `total_points`, `rebounds`, and `assists` in prospective players, as these were identified as the most powerful statistical indicators of longevity by the model.
5. **Consider Model Refinement:** For future iterations, explore more sophisticated models (e.g., Logistic Regression, Random Forests, Gradient Boosting) that do not assume feature independence and can capture more complex relationships. Additionally, incorporate more granular data if available (e.g., minutes played, advanced metrics, college stats, qualitative scouting reports) to enhance predictive power and provide a more holistic view.

In []:

In []:

In []: